

	<i>CAPE-Twiggs HSSSI-26 SlimSat Project</i> SlimSat Software Quick Start User's Guide	A9SP-2025- XB002 Rev A Draft
--	--	---

This document is a part of the CAPE-Twiggs HSSSI-26 SlimSat Project Documentation

PREPARED BY:

	Eric Tapio
	SFBA HSSSI-26 Space Mission Mentor

Record of Revisions				
REV	DATE	AFFECTED PAGES	DESCRIPTION OF CHANGE	AUTHOR(S)
A Draft	9/4/25	All	Initial Draft	Eric Tapio

Table of Contents

	Page
1 INTRODUCTION.....	3
2 REFERENCES, DOCUMENTS, & LINKS.....	3
2.1 Links	3
3 ACRONYMS AND DEFINITIONS	3
4 ITEMS REQUIRED	3
5 SLIMSAT SOFTWARE LIBRARY OVERVIEW	4
6 EXAMPLE SLIMSAT S/C BUS WITH PING PAYLOAD HARDWARE SETUP	5
7 EXAMPLE SLIMSAT S/C BUS WITH PING PAYLOAD SOFTWARE SETUP	6
7.1 SOFTWARE DOWNLOAD	6
7.2 SOFTWARE INSTALLATION.....	10
7.3 SOFTWARE VERICATION (USING ITSYBITSY NRF MICROCONTROLLER).....	13
7.4 SOFTWARE VERICATION (USING A DIFFERENT MICROCONTROLLER).....	15
7.5 SOFTWARE UPLOAD	17
7.6 RUNNING THE EXAMPLE CODE	18
7.7 SOFTWARE SOURCE FILE LOCATION	20
8 EXAMPLE SLIMSAT S/C BUS WITH PING PAYLOAD OPERATIONS CONCEPT.....	21

	<i>CAPE-Twiggs HSSSI-26 SlimSat Project</i> SlimSat Software Quick Start User's Guide	A9SP-2025- XB002 Rev A Draft
--	--	---

1 INTRODUCTION

This document introduces the HSSSI-26 SlimSat S/C Bus and Payload software, its structure, and the instructions on how to download, install and run the SlimSat code. Please note that the SlimSat S/C Bus and Payload code libraries are currently under development. Nevertheless, the libraries are available for use and user feedback is appreciated.

2 REFERENCES, DOCUMENTS, & LINKS

2.1 Links

Link #	Link Description	Link Title
1	SlimSat SW Github Repo	https://github.com/eric-tapio/HSSSI26_SlimSat
2	Arduino IDE Download	https://www.arduino.cc/en/software/
3	Ping Sensor	https://www.parallax.com/product/ping-ultrasonic-distance-sensor/
4	ItsyBitsy nRF Microcontroller	https://learn.adafruit.com/adafruit-itsybitsy-nrf52840-express

3 ACRONYMS AND DEFINITIONS

GS	Ground Station
H/W	Hardware
HSSSI	High School SlimSat Space Initiative
IDE	Integrated Development Environment
I/F	Interface
ITAR	International Traffic in Arms Regulations
LoRa	Long Range (Radio)
OOP	Object-Oriented Programming
P/L	Payload
S/C	Spacecraft
S/W	Software

4 ITEMS REQUIRED

Item #	Description	Qty	✓
1	Computer with Arduino IDE installed	1	
2	Microcontroller, such as the ItsyBitsy nRF	1	
3	USB A (or USB C) to Micro USB Cable/Adapter	1	
4	Ping Sensor	1	
5	Wireless Breadboard	1	
6	Jumper Wires	1	

5 SLIMSAT SOFTWARE LIBRARY OVERVIEW

This section provides a high-level overview of the HSSSI26 SlimSat software architecture. There are multiple SlimSat software libraries that provide the aggregate capability of the SlimSat S/C Bus and Payload. The SlimSat software libraries and their dependencies are identified below in Figure 1.

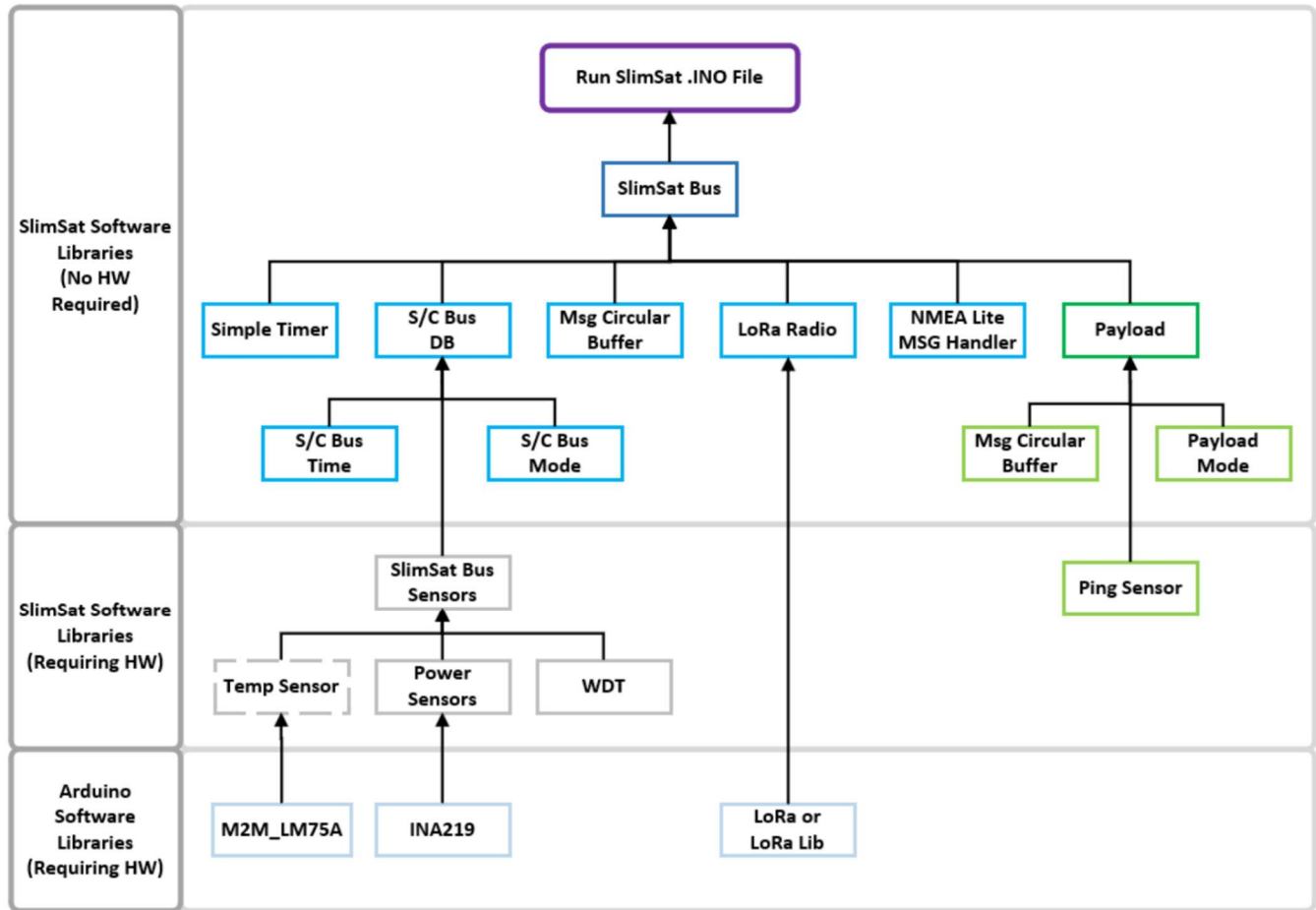


Figure 1 – HSSSI-26 SlimSat Software Class Architecture Diagram

The primary objective of this document is to provide instructions for downloading and running an example SlimSat S/C Bus and Payload example using the Ping Sensor. As a result, the scope of SlimSat libraries that will be used in the proceeding examples are the SlimSat software libraries that do not require hardware, except for the Ping Sensor library.

6 EXAMPLE SLIMSAT S/C BUS WITH PING PAYLOAD HARDWARE SETUP

This section shows the hardware setup that is required to run the SlimSat Sim software, which includes the Ping Sensor Payload.

To setup the SlimSat S/C Bus with Ping Sensor Payload Sim that will be uploaded and run, use the items required listed in Section 4, and attach them as shown below in Figure 2.



Figure 2 – SlimSat S/C Bus with Ping Payload Sim Example Hardware Setup

The wiring diagram for the Ping Sensor to the microcontroller is shown below in Figure 3.

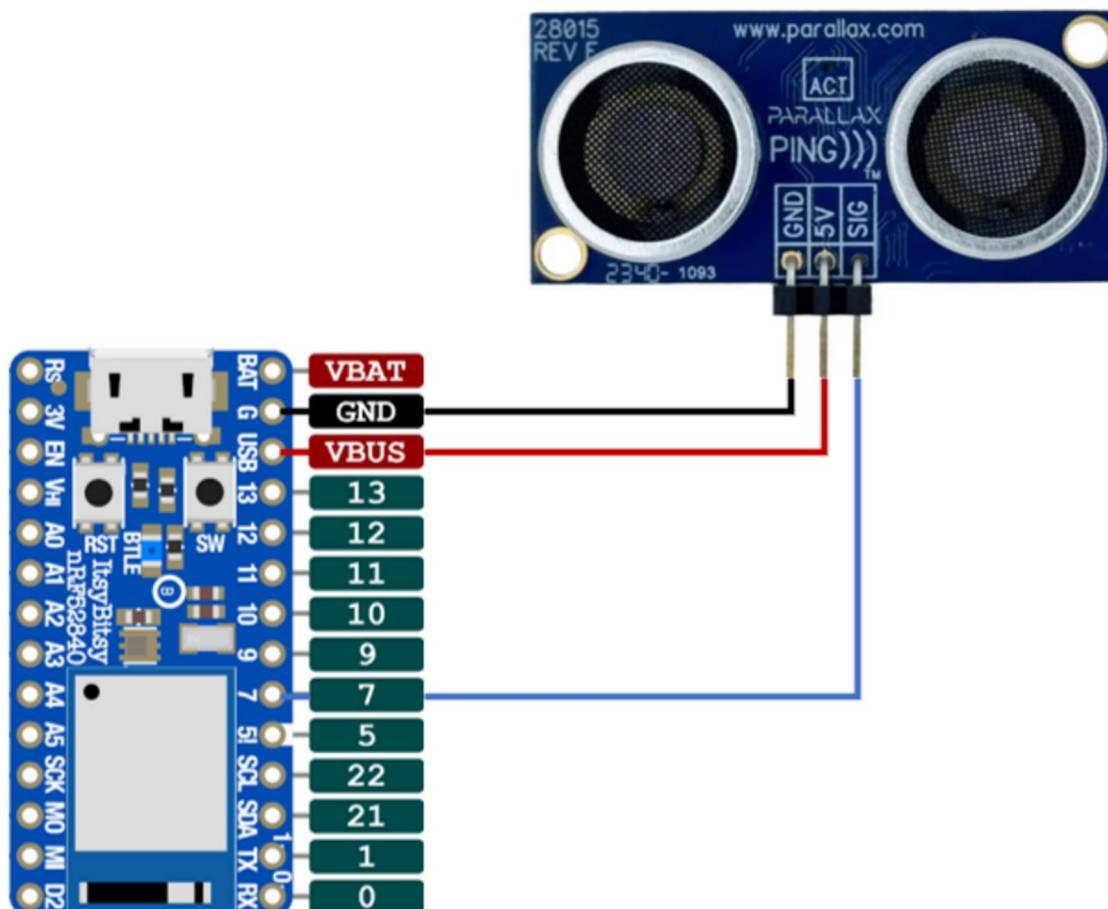


Figure 3 –Ping Sensor to Microcontroller Wiring Diagram

Note: This section assumes that the user is using the ItsyBitsy nRF microcontroller, which is currently baselined to be used on the SlimSat. The SlimSat Sim code presented in this document can run on nearly any microcontroller supported by the Arduino IDE. If a different microcontroller is used, ensure to connect the Ping Sensor SIG pin to digital pin 7 on the microcontroller.

7 EXAMPLE SLIMSAT S/C BUS WITH PING PAYLOAD SOFTWARE SETUP

This section describes the steps required to download, install, open, view, and upload the SlimSat Sim software, which includes the Ping Sensor Payload, to the microcontroller.

7.1 SOFTWARE DOWNLOAD

To download the SlimSat S/C Bus and Payload code from Github, in a web browser navigate to:
https://github.com/eric-tapio/HSSSI26_SlimSat

Note: For compliance with International Traffic in Arms Regulations (ITAR), the HSSSI26_SlimSat Github repo is a private repo, which is not visible nor available to the public. The HSSSI26

Payload repo, however, is a public repo, since its scope does not pertain to satellite or satellite systems.

Access to private Github repos must be provided by the repo manager. As a result, for access to the HSSSI26_SlimSat Repo, please contact Eric at eric.d.tapio@gmail.com

1) From the HSSSI26_SlimSat Github repo page, click on the green Code button, as shown below in Figure 4.

2) Then select "Download ZIP" from the dropdown menu that appears.

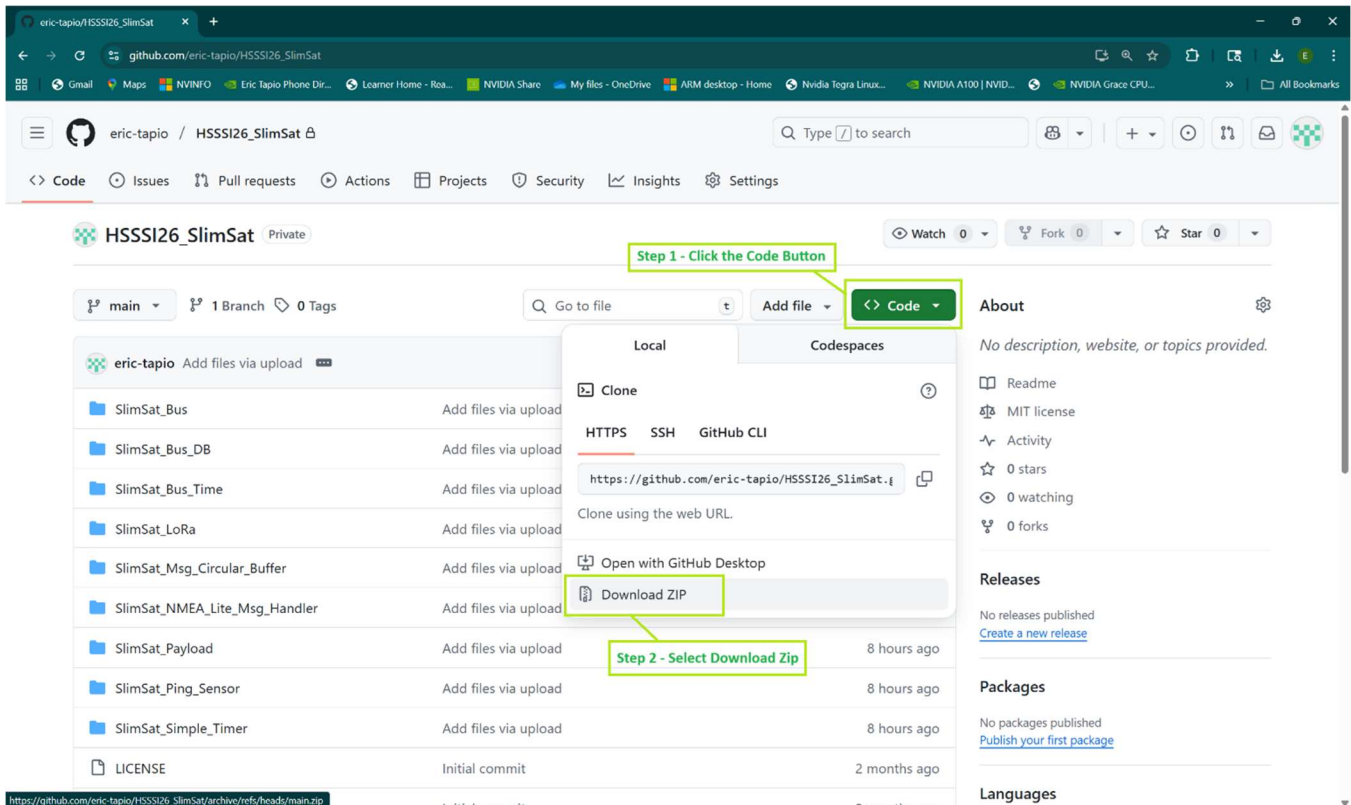


Figure 4 – Download the SlimSat Library Repository ZIP File

Using a file explorer app, navigate to the location of the downloaded HSSSI26_SlimSat-main ZIP file, as shown below in Figure 5.

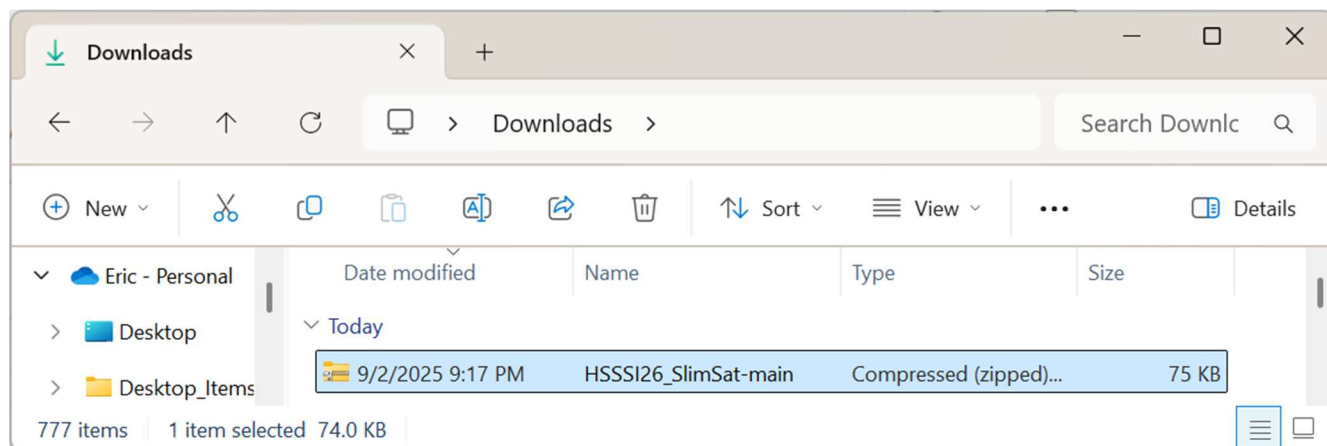


Figure 5 – Locate the Downloaded SlimSat Library Repository ZIP File

Then unzip the ZIP File. On a Windows machine, this can be performed by right-clicking on the file, and then choosing “Extract All”, as shown below in Figure 6.

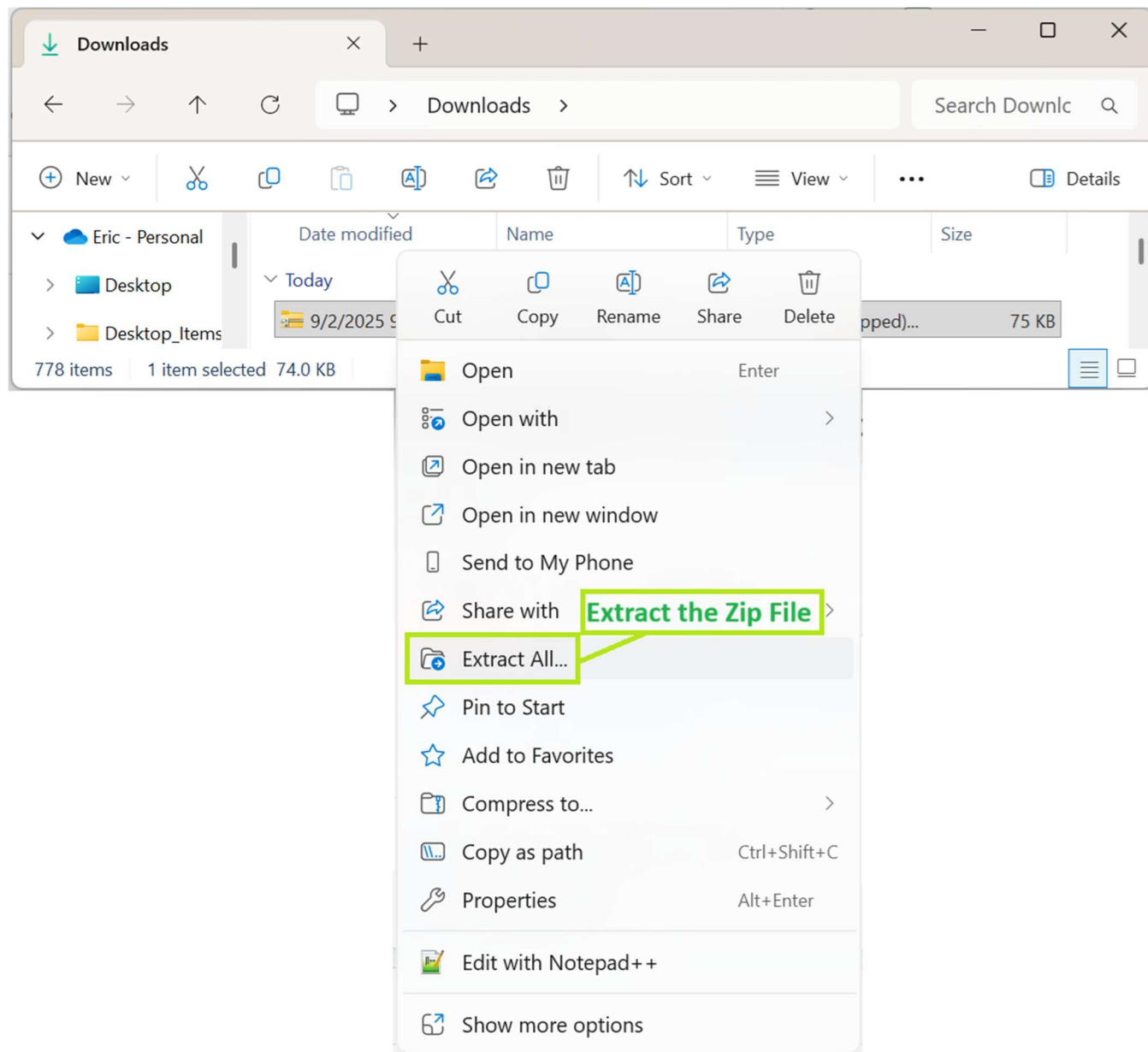


Figure 6 – Extract (Unzip) the SlimSat Library ZIP File

Next, open the extracted folder that contains the SlimSat libraries, which will be named HSSSI26_SlimSat-main, as shown below in Figure 7.

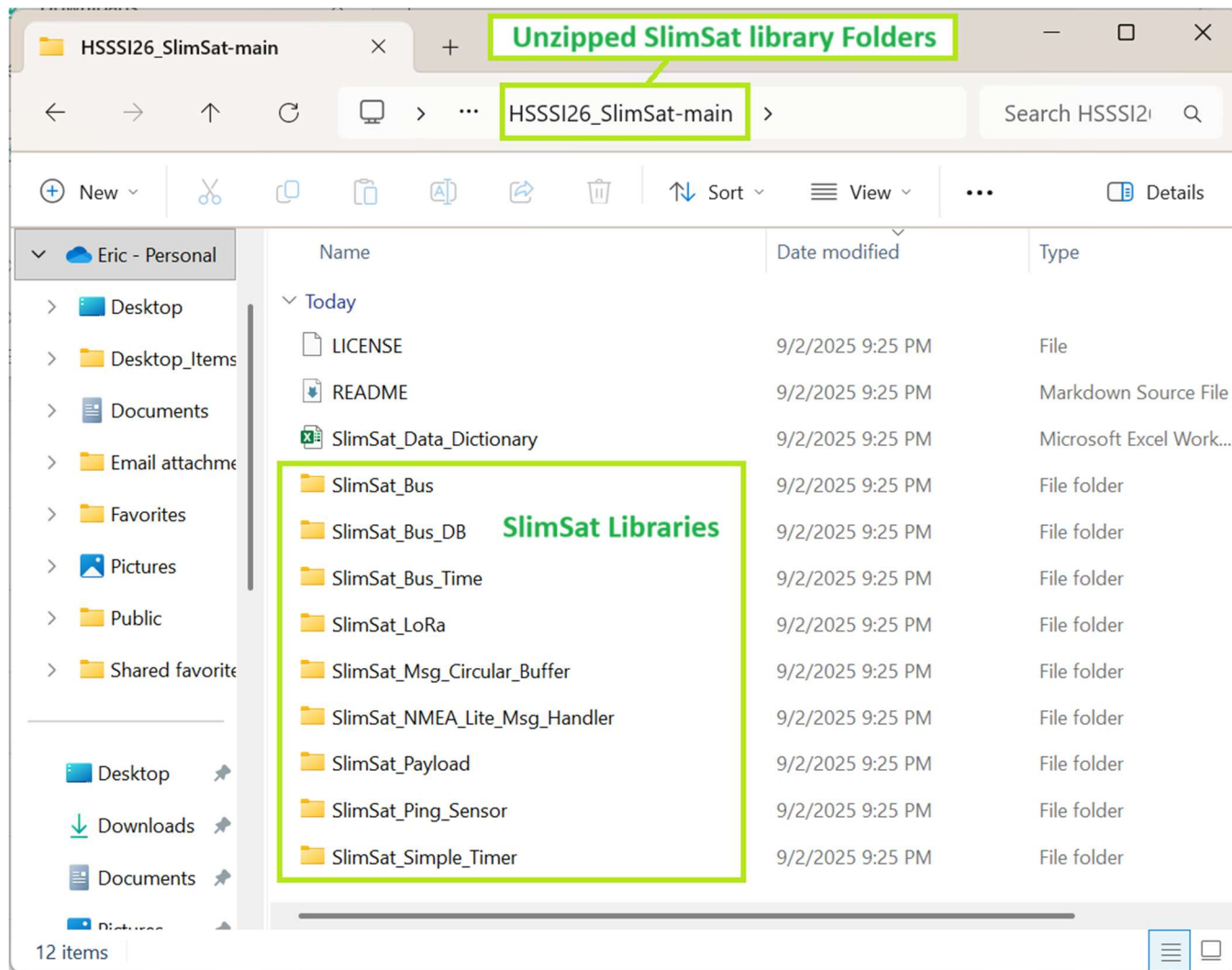


Figure 7 – View the Contents of the Extracted SlimSat Library Repository

7.2 SOFTWARE INSTALLATION

If you have not already done so, first download and install the Arduino Integrated Development Environment (IDE). The Arduino IDE can be downloaded from:
<https://www.arduino.cc/en/software/>

If you already have the Arduino IDE installed, close the application first before proceeding to the next step.

Next, to install the SlimSat S/C Bus and Payload code, use a new file explorer window and navigate to the Arduino “libraries” folder. The libraries folder can be found within the Arduino folder that is created when the Arduino IDE is installed. On a Windows machine, the common location for the Arduino libraries folder is shown in Figure 8.

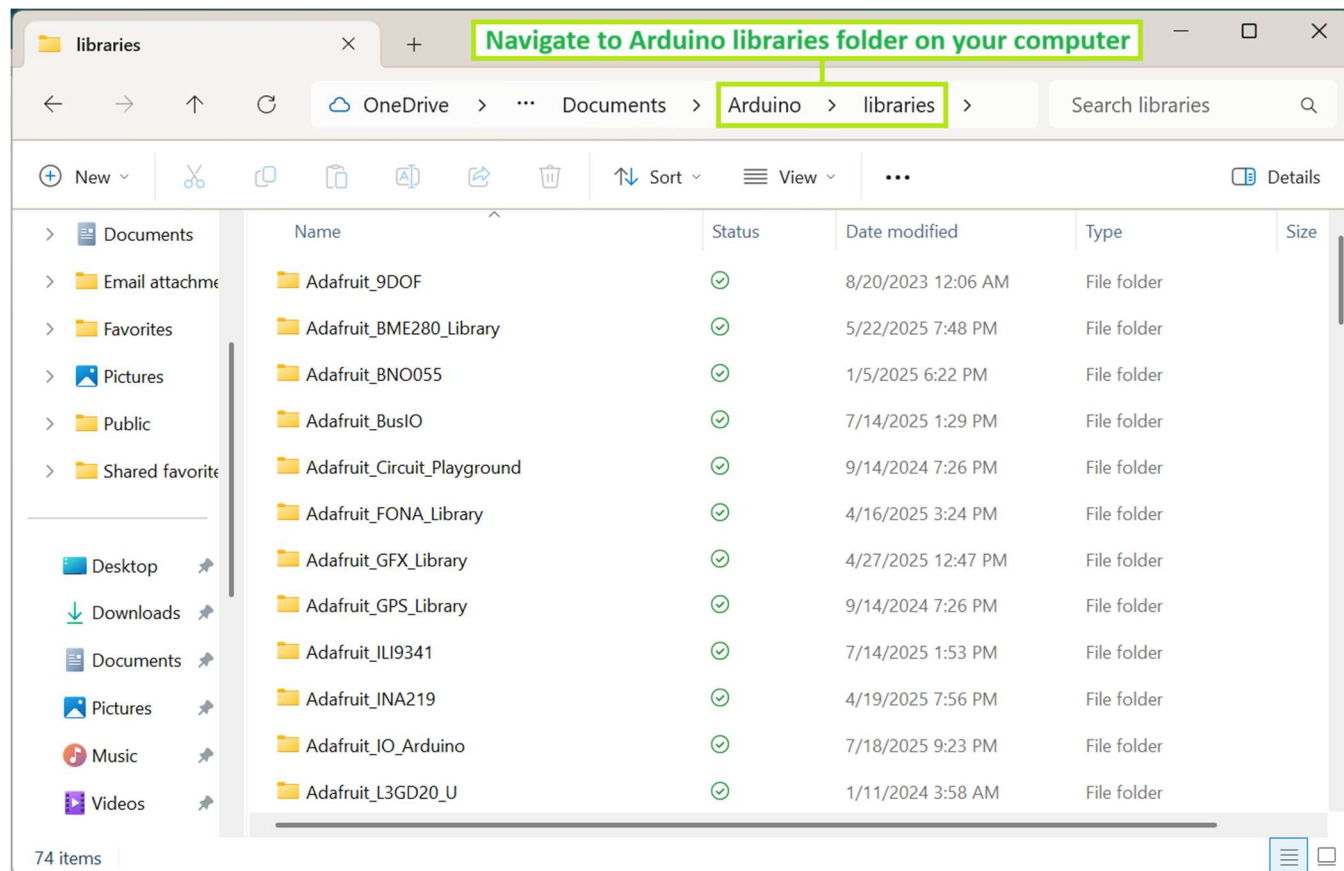


Figure 8 – Open the Arduino Libraries Folder in a new File Browser Window

Then copy the SlimSat library code files unzipped from HSSSI26_SlimSat-main to the Arduino libraries folder, as shown below in Figure 9.

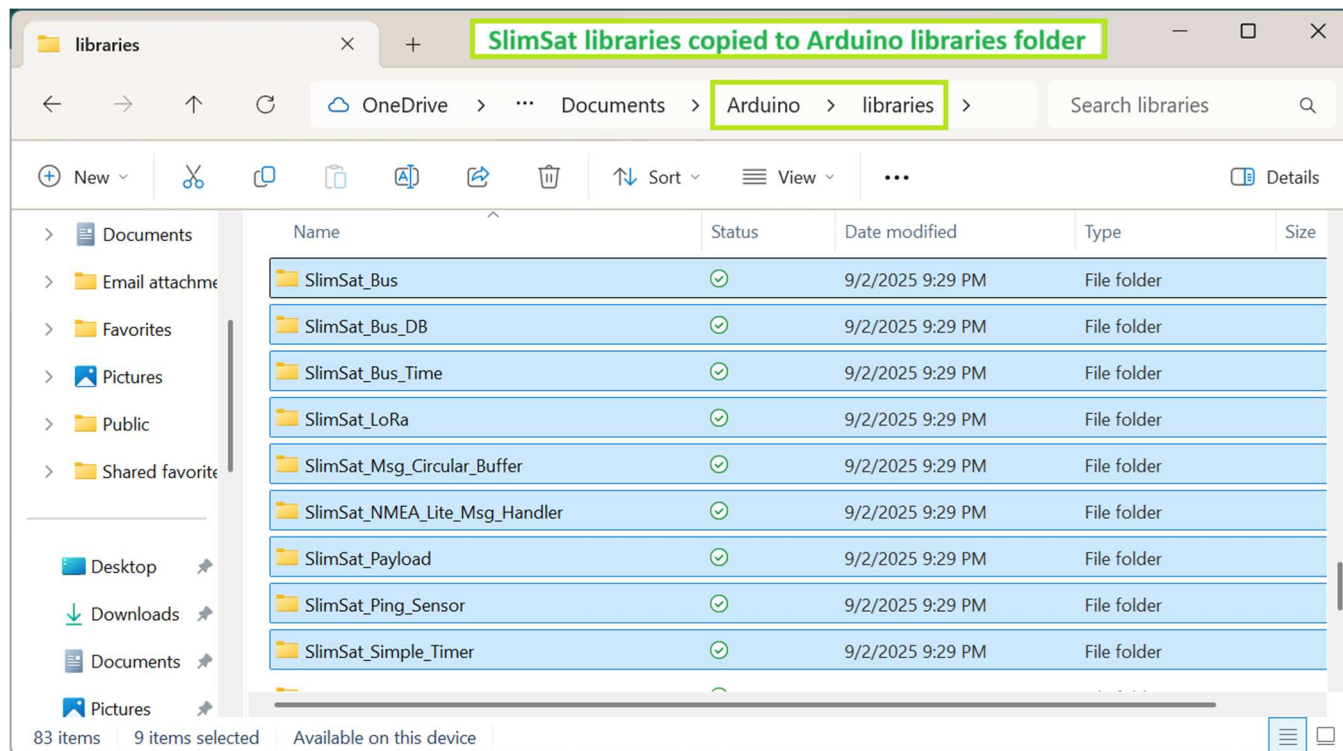


Figure 9 – Copy the SlimSat Library Folders to the Arduino Libraries Folder

Next, open the Arduino IDE.

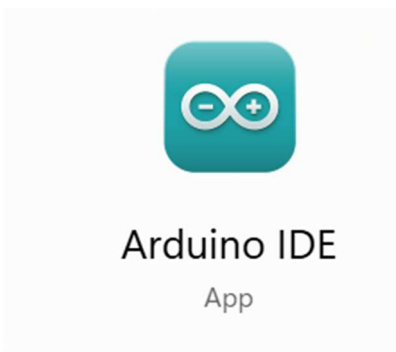


Figure 10 – Open the Arduino IDE

Upon opening, the Arduino IDE searches through the Arduino libraries folder and imports all libraries, including any examples provided in the imported libraries. Several of the HSSSI26 SlimSat and Payload libraries have example files included.

To open the HSSSI SlimSat and Payload examples, such as the Run SlimSat SW Sim example, in the Arduino IDE, click File -> Examples -> SlimSat SC Bus -> Run_SlimSat_SW_Sim, as shown below in Figure 11.

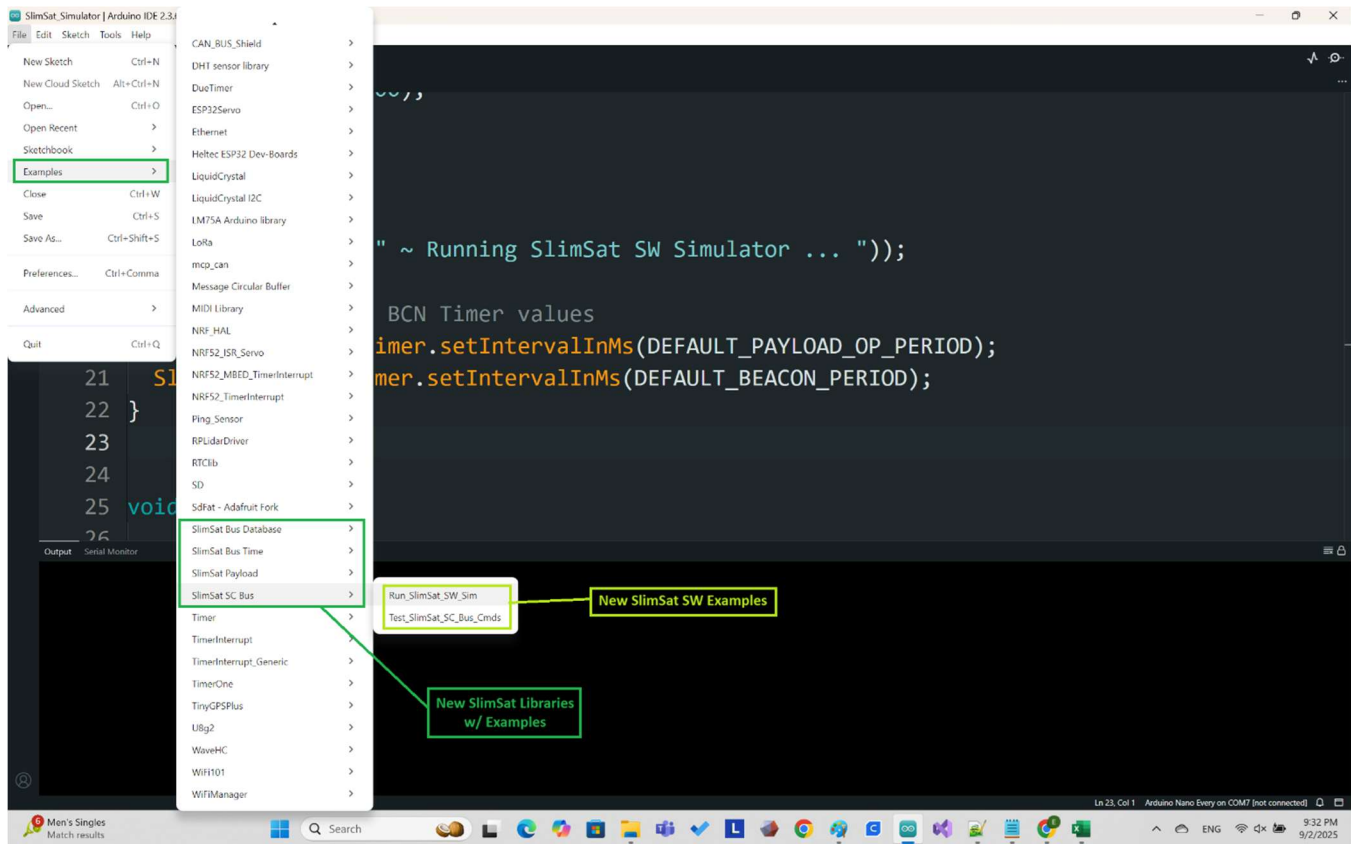


Figure 11 – In Arduino IDE, Click File -> Examples, & Navigate to Run SlimSat SW Sim Example

After clicking on Run_SlimSat_SW_Sim, the Run SlimSat SW Sim example INO file will open in a new window, such as that shown in Figure 12 in the next section. After being opened, the next step will be to verify the code prior to uploading the code to the microcontroller. The process of verifying and uploading the code is described in the next few sections.

7.3 SOFTWARE VERIFICATION (USING ITSYBITSY NRF MICROCONTROLLER)

Note: This section assumes that the user is using the ItsyBitsy nRF microcontroller, which is currently baselined to be used on the SlimSat. The Run_SlimSat_SW_Sim code can run on nearly any microcontroller supported by the Arduino IDE, however, at least one change to the INO file will be required first. If the user is using a microcontroller other than the ItsyBitsy nRF, then skip to Section 7.4.

The next step prior to uploading the Run_SlimSat_SW_Sim code is to verify the code, which will compile the code. To verify the code, in the Arduino IDE, click on the Verify icon, as shown below in Figure 12.

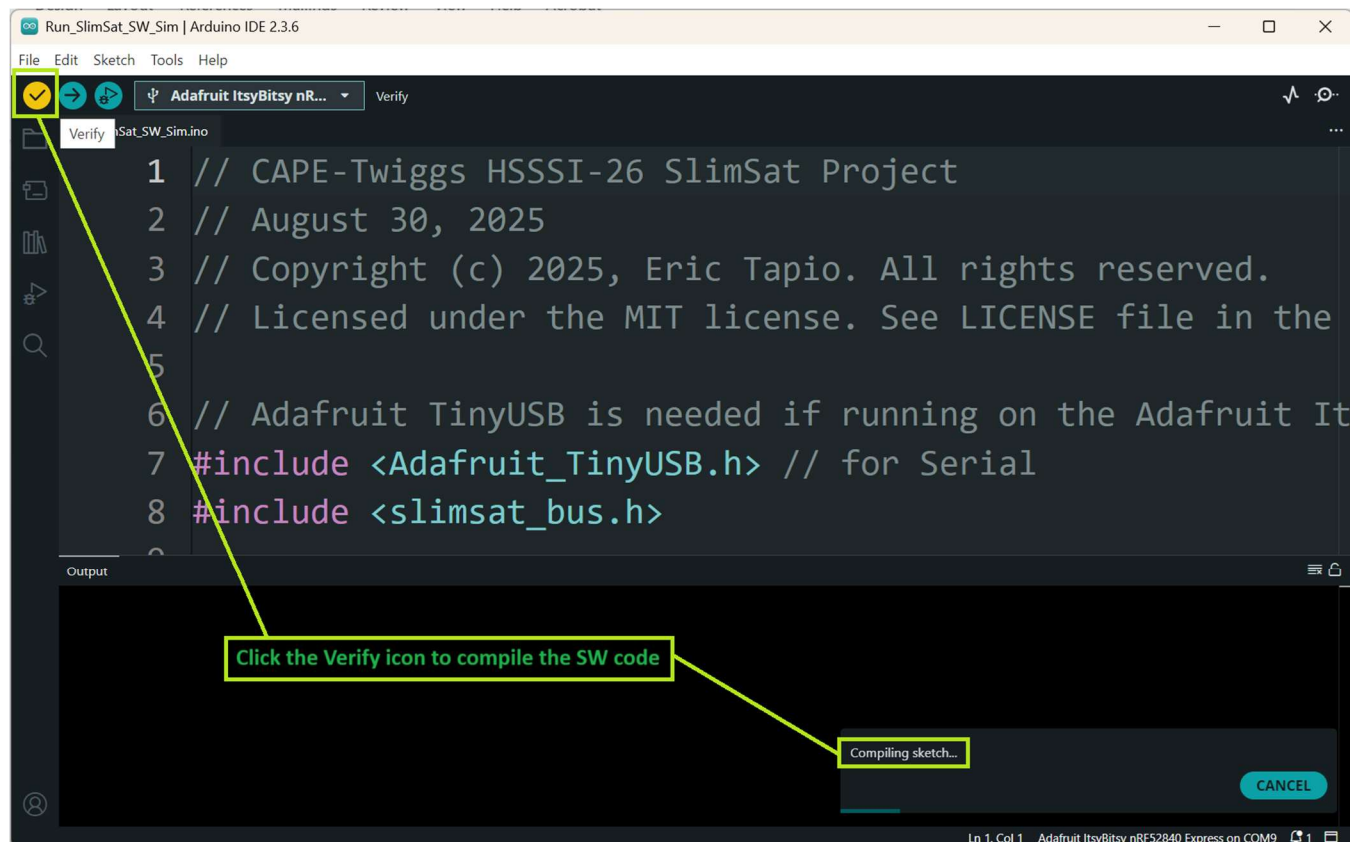


Figure 12 – Verify the Run SlimSat SW Sim Example

The Run_SlimSat_SW_Sim example code should compile successfully, and the user should see an output like that shown below in Figure 13.

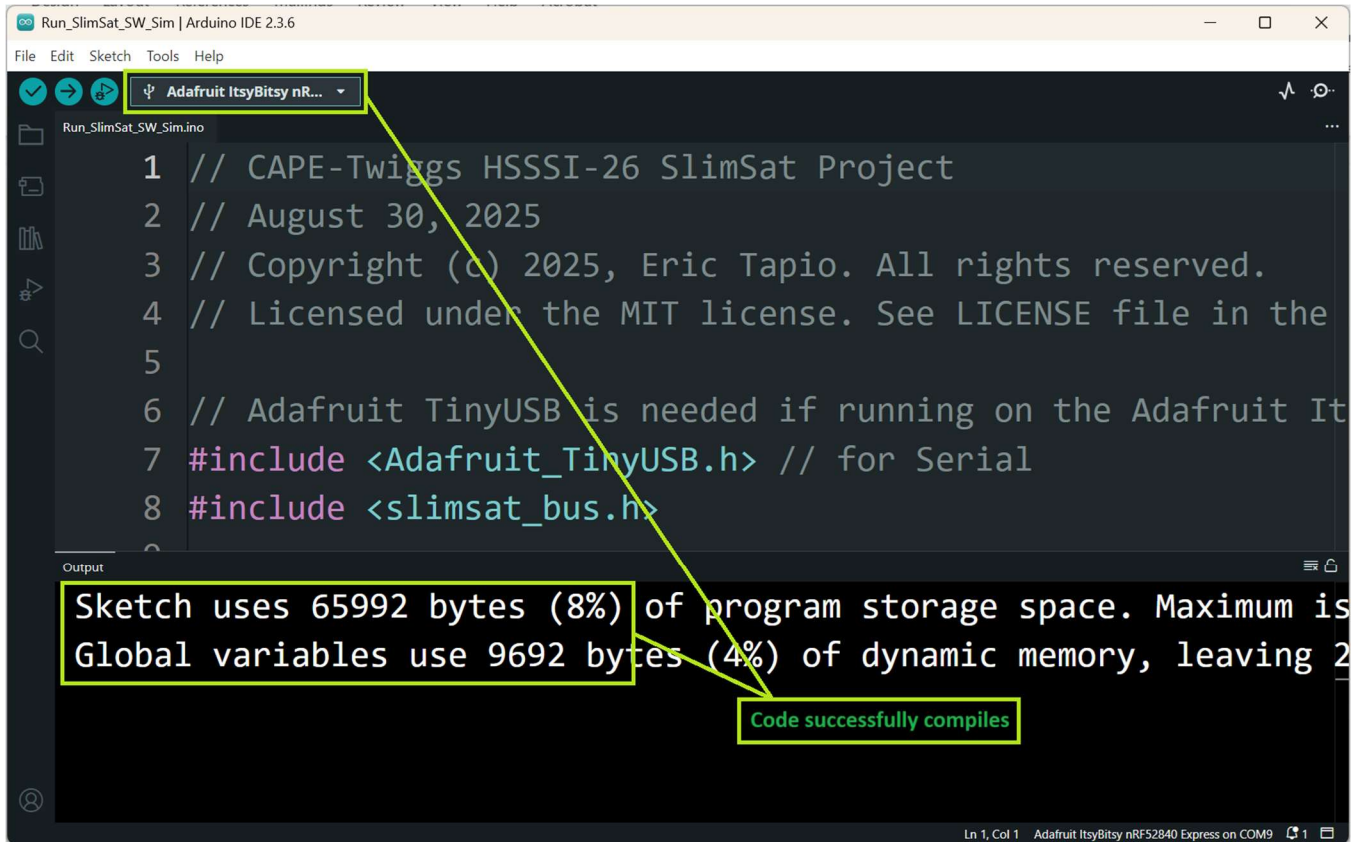


Figure 13 – Successful Verification of the Run SlimSat SW Sim Example

7.4 SOFTWARE VERIFICATION (USING A DIFFERENT MICROCONTROLLER)

If the user is not using the ItsyBitsy nRF microcontroller, then one modification to the example code is required.

Without the code modification, the error shown below in Figure 14 will likely be encountered during the code verification process.

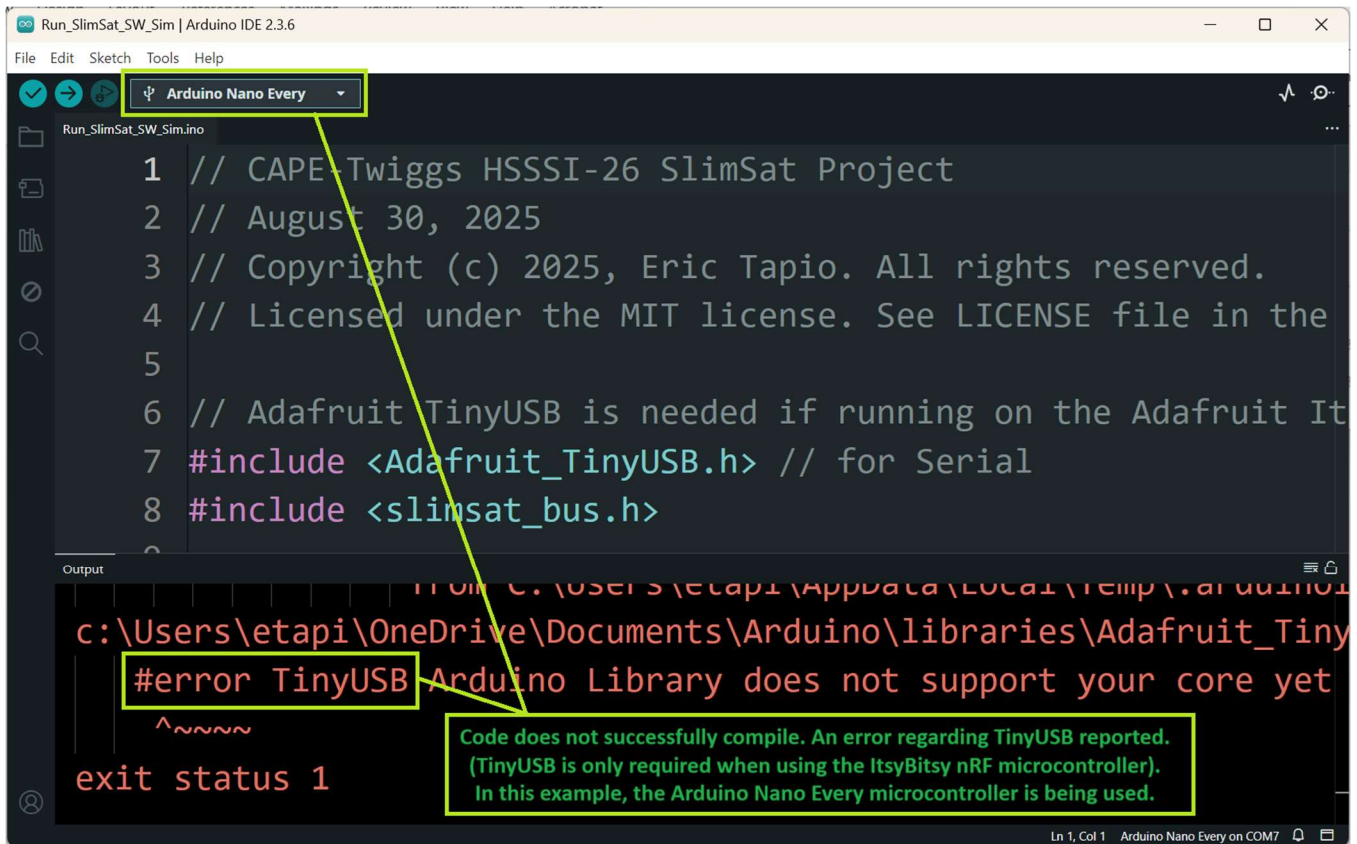


Figure 14 –TinyUSB Verification Error of the Run SlimSat SW Sim Example

The ItsyBitsy nRF microcontroller requires the TinyUSB library, and most other microcontrollers do not. If the user encounters the TinyUSB error as shown, the fix is to change the `#include <Adafruit_TinyUSB.h>` line of code to a comment, as shown below in Figure 15.

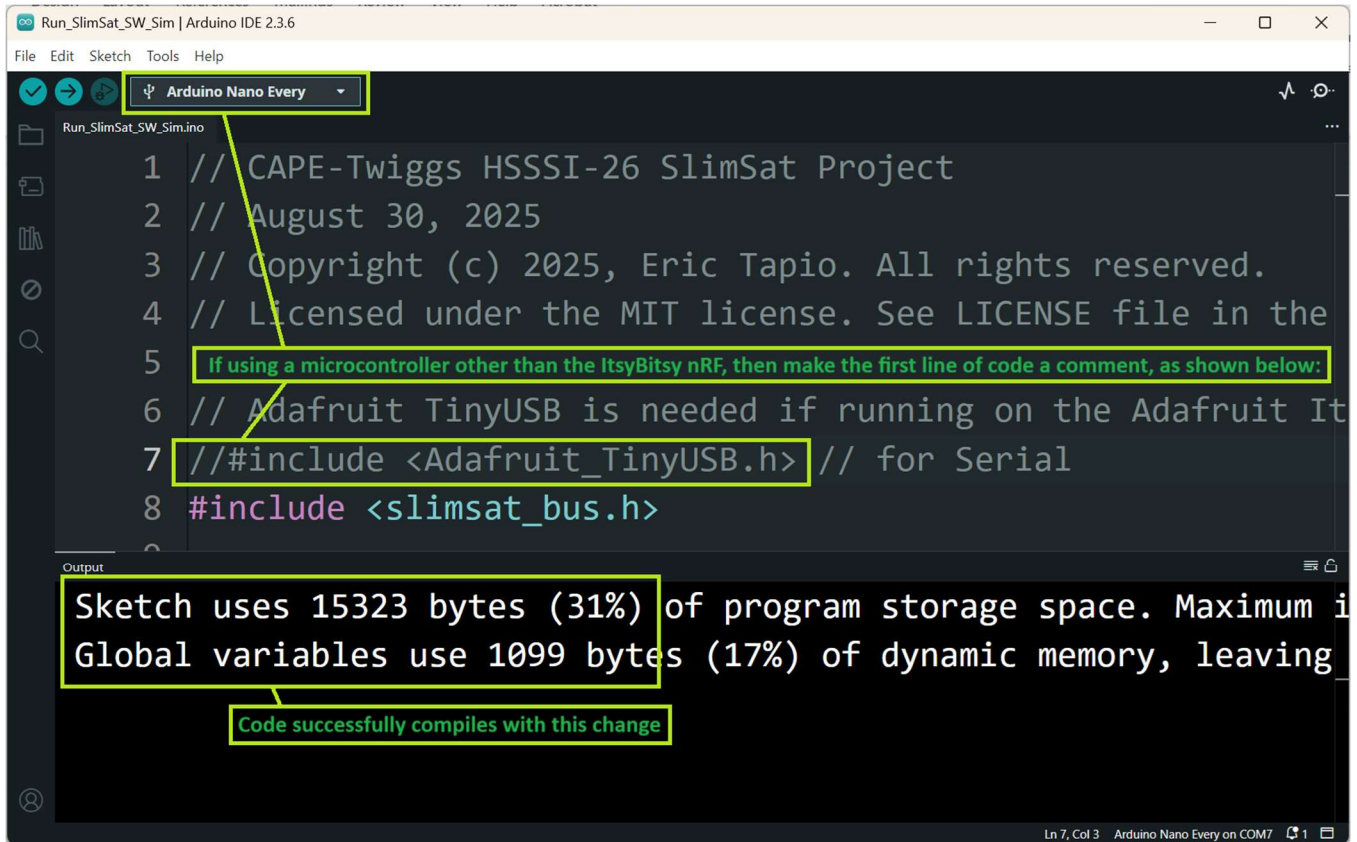


Figure 15 – Successful Fix and Verification of the Run SlimSat SW Sim Example

7.5 SOFTWARE UPLOAD

Now that the code has been successfully verified, regardless of which microcontroller the user is using, the next step of the process to run the Run_SlimSat_SW_Sim example is to upload the code.

To upload the Run_SlimSat_SW_Sim example code, click on the Arduino IDE Upload icon, as shown below in Figure 16.

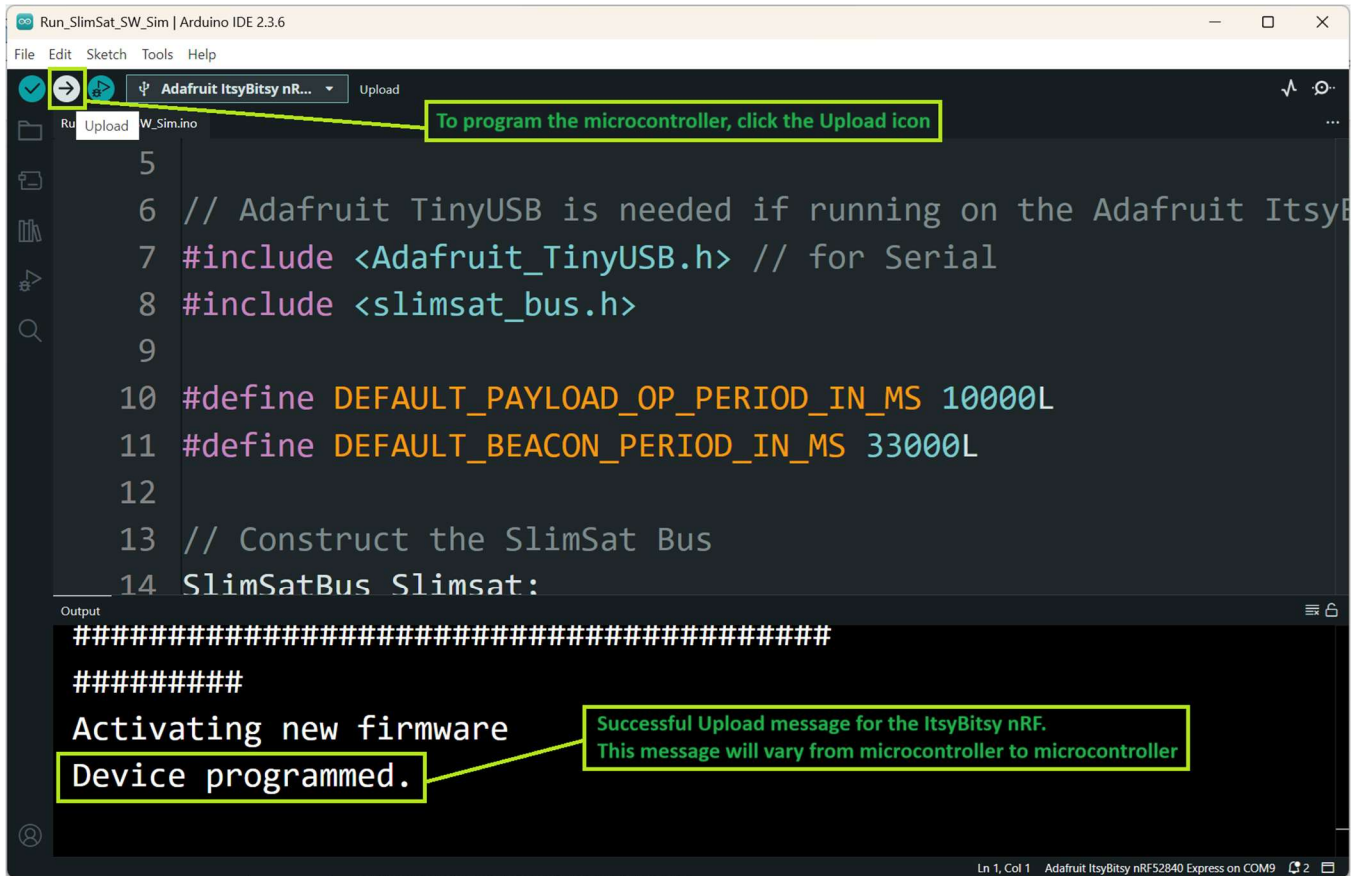


Figure 16 – Successful Upload of the Run SlimSat SW Sim Example

After a successful upload of code to the ItsyBitsy nRF, a message of “Device programmed.” should be displayed, as shown above in Figure 16. Note: the message to indicate a successful upload of code varies from microcontroller to microcontroller

7.6 RUNNING THE EXAMPLE CODE

After a successful upload of code to the microcontroller, the example code will automatically start running. To interact with Run_SlimSat_SW_Sim, the Arduino IDE Serial Monitor is used.

If the Arduino IDE Serial Monitor is not automatically displayed upon uploading the code, then click on the Serial Monitor icon, as shown below in Figure 17.

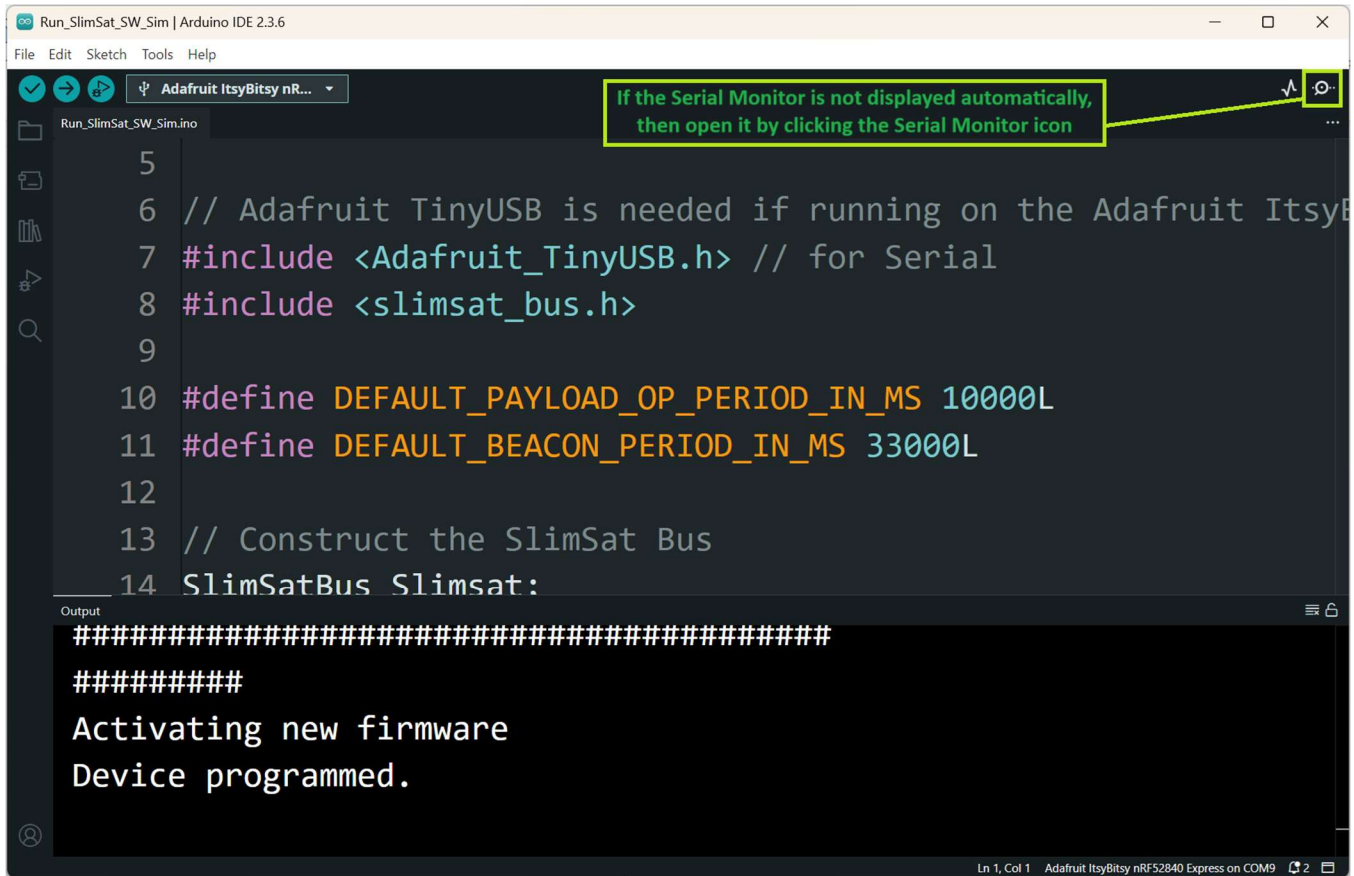


Figure 17 – Open the Arduino IDE Serial Monitor

From the Arduino IDE Serial Monitor is not automatically displayed upon uploading the code, then click on the Serial Monitor icon, as shown below in Figure 17.

The output from the Run_SlimSat_SW_Sim is displayed in the Serial Monitor window, as shown in Figure 18.

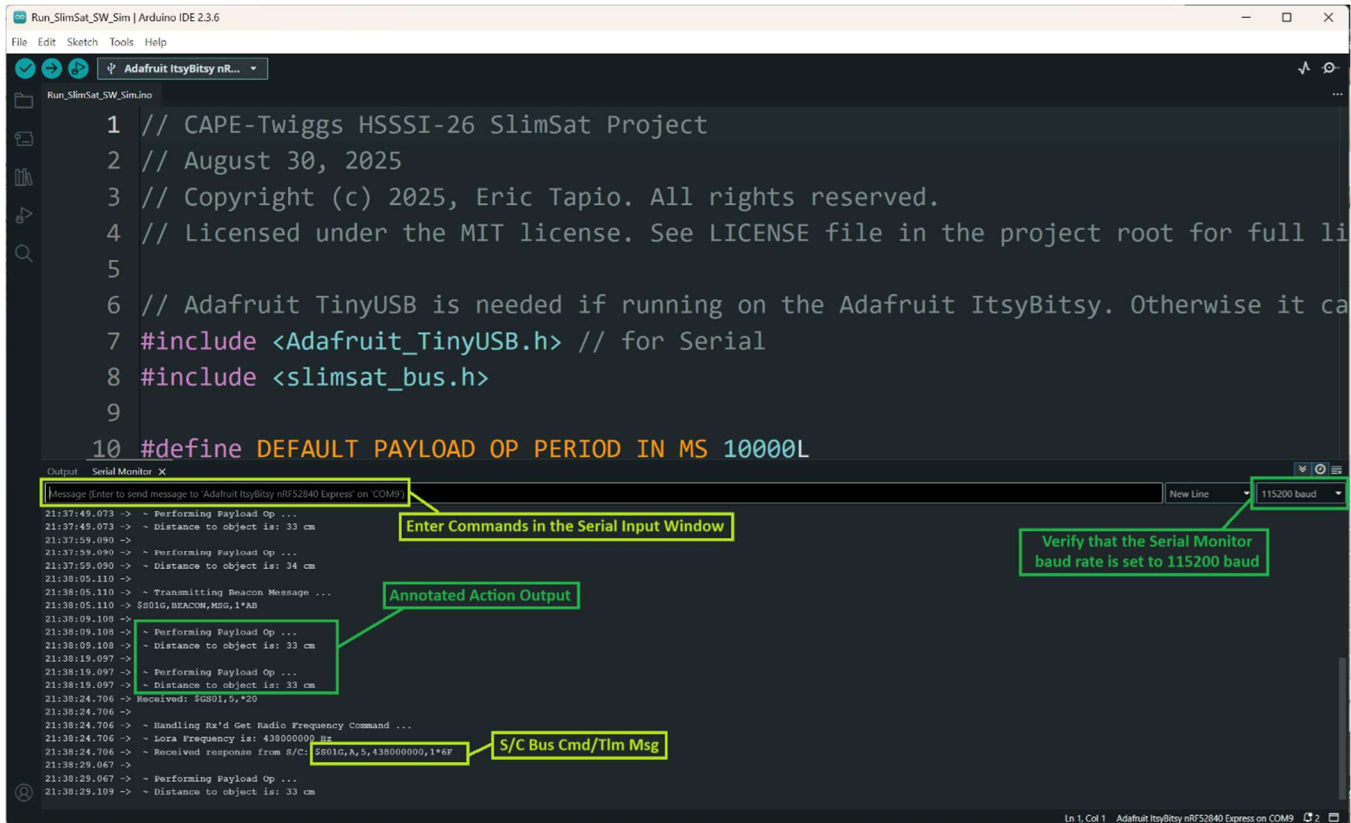


Figure 18 – Open the Arduino IDE Serial Monitor

If the Serial Monitor output is not readable characters, then ensure that the Serial Monitor data rate is set to 115200 baud. Input to the Run_SlimSat_SW_Sim can be entered in the Serial Monitor Input window, as highlighted above in Figure 18.

7.7 SOFTWARE SOURCE FILE LOCATION

Finally, to view SlimSat source files, navigate to the “src” folder in the SlimSat library that the user is interested in viewing. The location for the SlimSat Bus library source files, as an example, is shown below in Figure 19.

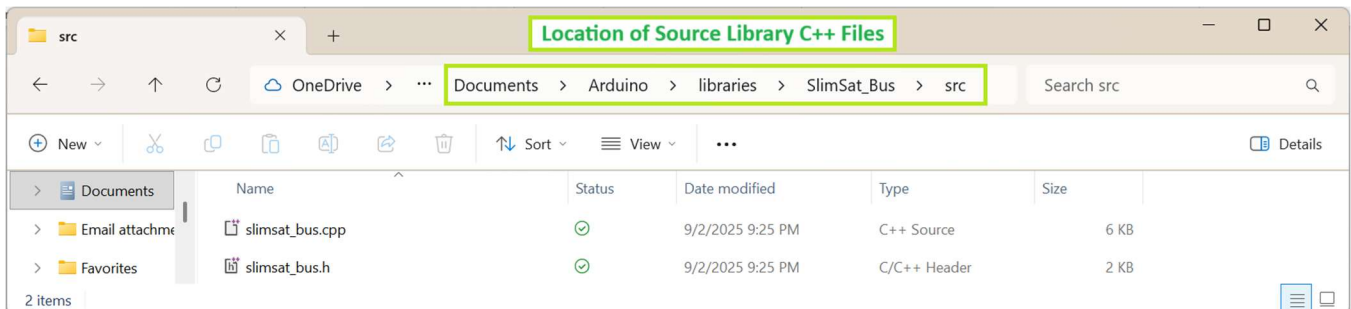


Figure 19 – Location of SlimSat Bus Library Source Files

8 EXAMPLE SLIMSAT S/C BUS WITH PING PAYLOAD OPERATIONS CONCEPT

This section describes the SlimSat S/C Bus with Ping Payload operations concept including command and telemetry format, command list, and example S/C Bus and Payload commands.

Context Reminder of the Development SlimSat System and How it operates:

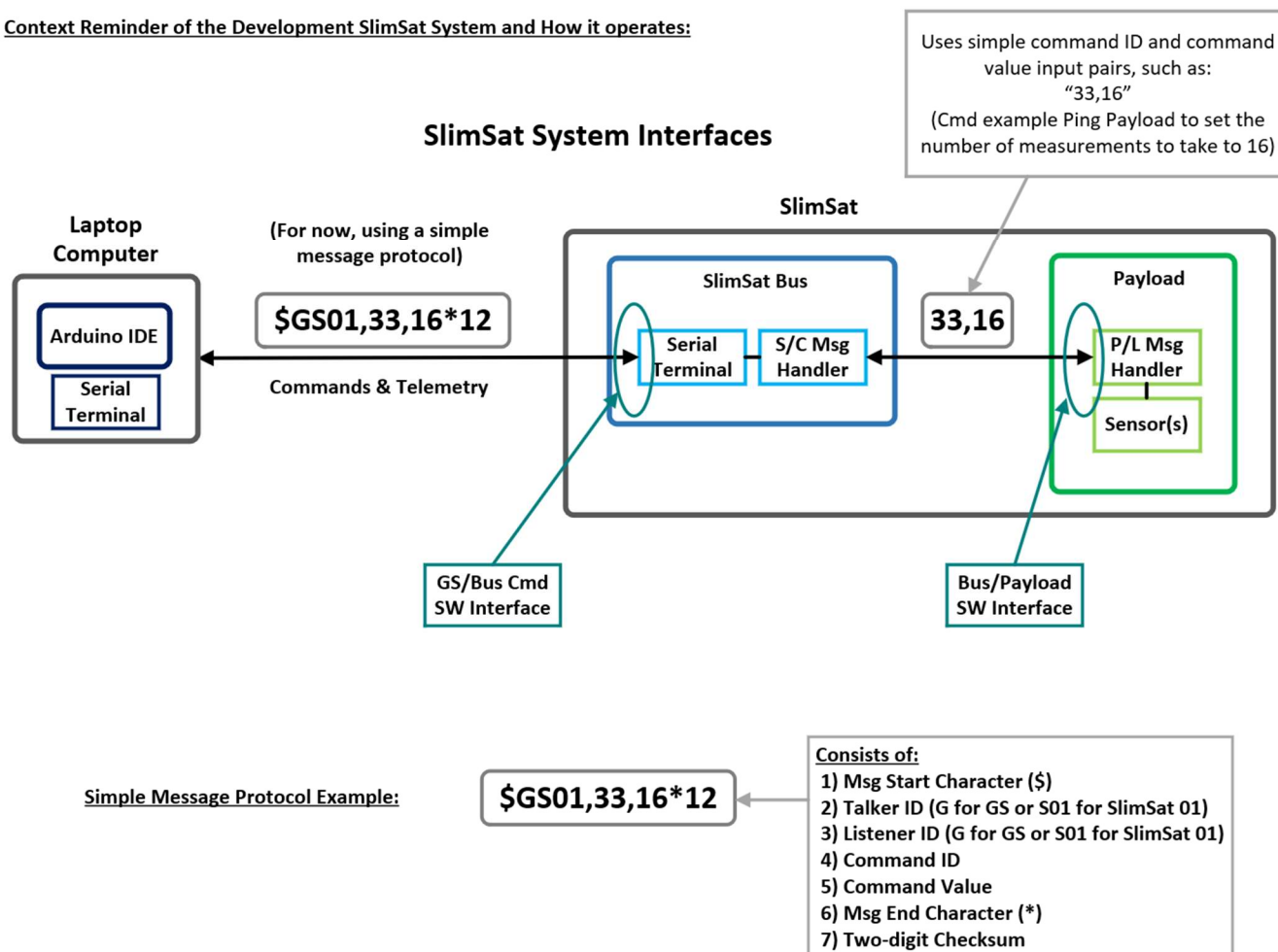


Figure 20 – SlimSat S/C Bus and Payload Operations Concept Overview

A high-level overview of the SlimSat S/C Bus and Payload operations concept is illustrated above in Figure 20.

The preliminary list of commands that can be sent to the S/C Bus are shown below in Table 1.

Table 1: SlimSat S/C Bus Command List

Command Description	Command ID	Command Value	Units	Example Command
Reserved	0	--	--	N/A
Ping	1	--	--	\$GS01,1,*24
Reboot	2	--	--	\$GS01,2,*27
Get Register Value	3	Register Number	--	\$GS01,3,1234*22
Set Register Value	4	Register Value	--	\$GS01,4,5678*2D
Get LoRa Frequency	5	--	Hz	\$GS01,5,*20
Set LoRa Frequency	6	Frequency Value	Hz	\$GS01,6,438123456*1B
Get LoRa Bandwidth	7	--	Hz	\$GS01,7,*22
Set LoRa Bandwidth	8	Bandwidth Value	Hz	\$GS01,8,30000*1E
Get LoRa Spread Factor	9	--	--	\$GS01,9,*2C
Set LoRa Spread Factor	10	Spread Factor Value	--	\$GS01,10,4*20
Get LoRa RSSI (Received Signal Strength Indicator)	11	--	dBm	\$GS01,11,*15
Get Beacon Period	12	--	ms	\$GS01,12,*16
Set Beacon Period	13	Period in Sec	ms	\$GS01,13,33000*27
Get S/C Bus Mode	14	--	--	\$GS01,14,*10
Set S/C Bus Mode	15	1 -> Safe 2 -> Nominal On	--	\$GS01,15,1*20
Get S/C Bus Epoch Time	16	--	ms	\$GS01,16,*12
Set S/C Bus Epoch Time	17	Epoch Time	ms	\$GS01,17,12345*22
Get S/C Bus Time	18	--	ms	\$GS01,18,*1C
Get S/C Bus Data	19	--	--	\$GS01,19,*1D
Get S/C Bus Data Stream Mode	20	--	--	\$GS01,20,*17
Set S/C Bus Data Stream Mode	21	0 -> Not Streaming 1 -> Stream Data	--	\$GS01,21,1*27
Get Payload Operation Period	22	--	ms	\$GS01,22,*15
Set Payload Operation Period	23	Period	ms	\$GS01,23,5000*11
Get Payload Mode	24	--	--	\$GS01,24,*13
Get Payload Data	25	--	--	\$GS01,25,*12

Additionally, a range of command IDs have been allocated to the Payload for SlimSat teams to use as they see fit for the design and operation of their payload.

The command list shown below in Table 2 are the Payload commands that were implemented for the example Ping Sensor Payload, which is a part of the Run_SlimSat_SW_Sim code described within this document.

Table 2: Example Ping Sensor Payload Command List

Ping Payload Example Commands	Command ID	Command Value	Units	Example Command
Ping Payload	30	--	--	\$GS01,30,*16
Get Payload State	31	--	--	\$GS01,31,*17
Get Number Measurements to Take	32	--	--	\$GS01,32,*14
Set Number Measurements to Take	33	Num Measurements	--	\$GS01,33,16*12
Take Round of Measurements	34	--	--	\$GS01,34,*12
Get Raw Measurements	35	--	cm	\$GS01,35,*13
Get Processed Measurement Data	36	--	cm	\$GS01,36,*10
Print Measurements	37	--	cm	\$GS01,37,*11
Optional Payload Command 9	38	--	--	\$GS01,38,*1E
Optional Payload Command 10	39	--	--	\$GS01,39,*1F